ApexNav Final Project 2026

Zero-shot Object Navigation with Semantic Maps, Memory, and Lightweight Models

> **Minimum Requirements**
>
> - GPU Memory: ≥ 16 GB
> - Disk Space: ≥ 100 GB

1. Project Introduction

This final project is built around the paper **ApexNav: An Adaptive Exploration Strategy for Zero-Shot Object Navigation with Target-centric Semantic Fusion** and its corresponding codebase.

ApexNav is a modular zero-shot object navigation system. Its main ideas include:

- **Adaptive Exploration**: switch between semantic-based exploration and geometry-based exploration
- **Target-centric Semantic Fusion**: maintain object-centered semantic memory across frames
- **Explicit Mapping**: construct frontier map, semantic score map, and fusion-based semantic memory
- **Waypoint-based Navigation**: use selected frontier or target waypoints to guide movement

This project does **not** require full reproduction of all results in the paper. Instead, students are expected to:

- understand the paper and system structure
- run a baseline on a small course-provided subset
- analyze system behavior through controlled experiments
- complete one selected project task

---

2. Project Options

All groups must complete the following common requirements:

Common Requirements

1. Read and understand the ApexNav paper
2. Run the baseline on the course-provided subset
3. Complete at least one controlled experiment
4. Choose **ONE** of the following project tasks

Project Tasks

- **Task 1: Localization with Semantic Maps**
- **Task 2: Multi-Target Memory Navigation**
- **Task 3: Integrated Semantic-Map Localization + Memory Navigation**
- **Task 4: Model Lightweighting and Deployment on LIMO**

---

3. Common Baseline Requirements

Before starting the selected project task, every group must complete the baseline stage.

Baseline Objectives

1. Understand the main modules in ApexNav
2. Map paper modules to code modules
3. Run the baseline on a small evaluation subset
4. Report baseline metrics and failure cases

Minimum Baseline Deliverables

- Paper summary
- System diagram
- Paper-module ↔ code-module mapping
- Baseline result table
- At least 3 success cases
- At least 3 failure cases

Minimum Baseline Metrics

- Success Rate (SR)
- SPL
- Average steps or average path length

Recommended Baseline Procedure

Step 1: Read the paper and identify the main pipeline
Step 2: Locate key modules in the codebase
Step 3: Run the baseline on a small subset
Step 4: Save logs, metrics, and qualitative cases
Step 5: Analyze failure patterns and choose one project task

---

4. Task 1: Localization with Semantic Maps

4.1 Introduction

In this task, students will use the semantic maps constructed by ApexNav to build a **coarse localization module**.
The system should not only search for target objects, but also estimate where the agent currently is based on current observations and the accumulated semantic map.

4.2 Project Description

Implement a semantic-map-based localization system on top of ApexNav.

4.3 Inputs

Possible inputs include:

- occupancy map
- explored / unexplored regions
- frontier map
- semantic score map
- object detections
- target-centric fusion memory
- current or historical observations
- optional short-horizon motion information

4.4 Outputs

Students should implement at least one of the following:

- room-level localization
- region-level localization
- top-k candidate localization

4.5 Requirements

1. Reproduce the baseline system
2. Design a global semantic-map representation and a local observation representation
3. Implement at least one localization baseline
4. Add one improved method
5. Visualize localization results
6. Evaluate using at least two metrics

4.6 Recommended Approach

Step 1: Extract a global semantic representation from the maps built by ApexNav

Step 2: Encode the current observation as a local semantic descriptor

Step 3: Build a simple baseline such as:

- semantic histogram matching
- room-cue matching
- top-k candidate region retrieval

Step 4: Improve the baseline with:

- multi-frame fusion
- motion prior
- confidence-weighted matching

Step 5: Visualize and evaluate the localization output

4.7 Suggested Evaluation Metrics

- room classification accuracy

- top-k localization recall
- region localization accuracy
- coarse localization error

4.8 Minimum Deliverables

- localization method description
- localization visualization results
- quantitative evaluation table
- failure case analysis

---

5. Task 2: Multi-Target Memory Navigation

5.1 Introduction

In this task, students will extend ApexNav from single-target navigation to **multi-target sequential navigation with memory reuse**.

5.2 Project Description

Implement a system that can handle multiple target-object requests in sequence and reuse previous observations, object memory, or semantic maps.

5.3 Inputs

- target sequence
- current observations
- current map
- previously stored object memory
- optionally, region memory

5.4 Outputs

Students should implement:

- a multi-target navigation pipeline
- optionally, an explicit object memory table or reusable semantic map

5.5 Requirements

1. Reproduce the baseline system
2. Implement a no-memory baseline
3. Implement at least one memory-based version
4. Compare memory vs no-memory performance
5. Analyze success and failure cases

5.6 Recommended Approach

Step 1: Build a baseline that resets search for each new target
Step 2: Build a simple memory table that stores observed object positions

Step 3: Reuse stored memory when the same category becomes the next goal
Step 4: Extend memory from object-level to map-level if possible
Step 5: Evaluate multi-target episodes and compare with the reset-search baseline

5.7 Suggested Evaluation Metrics

- multi-target success rate
- number of targets reached
- average steps per target
- average path length per target
- efficiency gain from memory reuse
- revisit cost reduction

5.8 Minimum Deliverables

- multi-target baseline
- memory-based method
- quantitative comparison table
- one successful memory-reuse case
- one failure case caused by wrong or incomplete memory

---

6. Task 3: Integrated Semantic-Map Localization + Memory Navigation (Advanced)

6.1 Introduction

This advanced task combines semantic-map localization, reusable semantic memory, and multi-target object navigation in one system.

6.2 Project Description

Design a unified extension in which:

- semantic-map localization helps estimate the current region or candidate locations
- explicit memory stores previously seen object instances
- later targets reuse both localization outputs and object memory

6.3 Requirements

1. Complete the common baseline requirements
2. Implement a semantic-map localization component
3. Implement a memory-based multi-target navigation component
4. Connect localization outputs to later target search
5. Compare against simpler baselines
6. Conduct failure analysis

6.4 Recommended Approach

Step 1: Build a semantic representation for localization on the explored map
Step 2: Store observed object instances and their map positions
Step 3: When a new goal appears, first estimate the current location or candidate regions from the semantic map, then use memory
Step 4: Compare with:

- no-memory baseline
- object-memory-only baseline
- localization-only baseline
  Step 5: Analyze which component contributes most

6.5 Suggested Evaluation Metrics

- multi-target success rate
- localization accuracy
- average steps per target
- search efficiency gain
- memory reuse benefit
- failure category breakdown

6.6 Minimum Deliverables

- integrated system design
- quantitative comparison across multiple baselines
- qualitative visualization
- failure analysis
- ablation of localization module vs object memory

---

7. Task 4: Model Lightweighting and Deployment on LIMO

7.1 Introduction

The original ApexNav stack may be too heavy for course machines and is not directly suitable for deployment on a small mobile robot.
In this task, students will first build a lightweight version of the ApexNav pipeline and then deploy the selected system on the **LIMO mobile platform**.

7.2 Project Description

Students must replace one or more heavy components with smaller models, rules, or lower-cost scheduling strategies while keeping the overall pipeline as intact as possible, and then deploy the resulting system on LIMO for real-world testing.

7.3 Possible Lightweighting Directions

- LLM lightweighting or rule-based replacement
- semantic scoring model lightweighting
- detection model lightweighting
- reduced segmentation frequency

- system-level scheduling optimization
- lower input resolution / half precision / on-demand model calls

7.4 Deployment Goal

The final lightweight system should be able to run on the LIMO platform and support at least one object-navigation-related behavior, such as:

- target-oriented search in a small indoor area
- waypoint following with semantic perception
- semantic-map-assisted localization or target approach
- memory-assisted revisit to a previously observed object

7.5 Requirements

1. Identify the major resource-consuming modules in the original system
2. Replace at least one module with a lighter alternative
3. Compare the original and lightweight configurations
4. Deploy the lightweight version on LIMO
5. Run at least one real-world demonstration task
6. Report both resource metrics and deployment performance
7. Explain the resource–performance–deployability tradeoff clearly

7.6 Recommended Approach

Step 1: Measure the original configuration on the development machine
Step 2: Choose one or more modules to replace
Step 3: Keep the interfaces as similar as possible
Step 4: Build a lightweight end-to-end version that can run stably
Step 5: Integrate sensors, perception, and motion execution on LIMO
Step 6: Test the system in a small real-world environment
Step 7: Compare simulation results and real-world deployment results

7.7 Minimum Evaluation Metrics

Resource Metrics

- peak GPU / CPU memory
- average inference time
- runtime stability
- model size or deployment footprint

Task Metrics

- Success Rate (SR)
- average task completion time
- average path length or control steps
- if applicable: localization accuracy, target-approach success rate, revisit success rate

Deployment Metrics

- real-world task success rate on LIMO
- end-to-end latency
- control smoothness / intervention frequency
- failure cases during deployment

7.8 Minimum Deliverables

- original model-stack description
- lightweight replacement design
- resource comparison table
- deployment system diagram for LIMO
- real-world test setup description
- quantitative deployment results
- demo video on LIMO
- resource–performance–deployment tradeoff discussion

---

8. Recommended Project Steps

Step 1: Read the paper

- understand the overall pipeline
- identify the key modules
- summarize the main ideas

Step 2: Read the code

- find the main entry
- locate mapping, exploration, fusion, and navigation modules
- complete the paper-to-code mapping

Step 3: Run the baseline

- test the system on the course subset
- record metrics and cases
- analyze failure patterns

Step 4: Choose one project task

- Task 1: Localization with Semantic Maps
- Task 2: Multi-Target Memory Navigation
- Task 3: Integrated system
- Task 4: Model Lightweighting and Deployment on LIMO

Step 5: Build the task baseline

- design a simple baseline for the selected task
- make sure it can be evaluated fairly

Step 6: Implement your improved method

- add one reasonable extension
- keep the comparison controlled

Step 7: Implement the selected task

- build the final system for the chosen task
- keep the comparison controlled

Step 8: Organize results

- prepare report
- prepare slides
- prepare demo video
- submit runnable code and documentation

---

9. Grading

Component	Task 1	Task 2	Task 3
Baseline reproduction	20	20	15
Task design and implementation	25	25	25
Controlled experiments	15	15	15
Model lightweighting	15	15	15
Report quality	15	15	15
Code / documentation	10	10	10
Innovation bonus	-	-	5

---

10. Resources

Core Project

1. **ApexNav (Code)**: https://github.com/SYSU-STAR/ApexNav
2. **ApexNav (Paper / arXiv)**: https://arxiv.org/abs/2504.14478

Embodied AI / Navigation Frameworks

1. **Habitat-Lab**: https://github.com/facebookresearch/habitat-lab
2. **Habitat-Sim**: https://github.com/facebookresearch/habitat-sim
3. **AI Habitat Website**: https://aihabitat.org/

Datasets

1. **HM3D Dataset**: https://aihabitat.org/datasets/hm3d/

2. **Matterport3D Dataset**: https://niessner.github.io/Matterport/
3. **HM3D Semantics**: https://aihabitat.org/datasets/hm3d-semantics/

Related Projects / Codebases

1. **SemExp (Goal-Oriented Semantic Exploration)**: https://github.com/devendrachaplot/Object-Goal-Navigation
2. **SemExp Project Page**: https://devendrachaplot.github.io/projects/semantic-exploration
3. **MultiON (Code)**: https://github.com/saimwani/multiON
4. **MultiON Project Page**: https://shivanshpatel35.github.io/multi-ON/

Lightweight Models / Practical Tools

1. **MobileSAM**: https://github.com/chaoningzhang/mobilesam
2. **CLIP**: https://github.com/openai/CLIP
3. **GroundingDINO**: https://github.com/IDEA-Research/GroundingDINO
4. **YOLOv7**: https://github.com/WongKinYiu/yolov7

Mobile Platform

1. **LIMO Robot**: https://www.agilex.ai/products/limo
2. **LIMO ROS Documentation**: https://github.com/agilexrobotics/limo_ros

Key Papers

1. **ApexNav: An Adaptive Exploration Strategy for Zero-Shot Object Navigation with Target-centric Semantic Fusion**
2. **Object Goal Navigation using Goal-Oriented Semantic Exploration (SemExp)**
3. **MultiON: Benchmarking Semantic Map Memory using Multi-Object Navigation**
4. **Mapping High-level Semantic Regions in Indoor Environments without Object Recognition**
5. **QueSTMaps: Queryable Semantic Topological Maps for 3D Scene Understanding**
6. **Semantic Mapping in Indoor Embodied AI: A Survey**

---

11. Final Deliverables

1. **Code**
2. **Technical Report**
3. **Presentation Slides**
4. **Demo Video**

Suggested Report Structure

1. Introduction
2. Related Work / Background
3. Method
4. Experiments
5. Results
6. Failure Analysis

7. Conclusion
8. References